

# Silvaco ATHENA .str / .sta Format Specification

**Alec Haines**

alechaines561@gmail.com

<https://github.com/ahaines561>

ComponentSilvaco Interface for Garfield++

June 25, 2026

## 1 Overview

A Silvaco `.str` file is a line-oriented ASCII format. Each line begins with a single-character tag identifying the record type. Unlike Synopsys DF-ISE, the file does not use nested `Section { ... }` blocks; it is a flat sequence of tagged records whose counts are declared up front.

Crucially for Garfield++: the mesh topology is fully present. Triangular elements reference vertex indices directly (no edge indirection as in DF-ISE), so a complete FEM mesh (vertices + elements + regions) can be reconstructed without any Delaunay re-triangulation.

The file may be 2D (`Coord2D`) or 3D; this specification documents the 2D case. Structure (geometry + doping) and solution data (e.g., `ElectricField`) may be split across separate files (`.str` and `.sta`) sharing the same mesh.

## 2 Line Tags

The order in the file is, roughly: `v` → `j` → `k` → `m` → `c...` → `e...` → `t...` → `r + b...` (per region) → `s` → `n...`

| Tag                        | Record Type          | Count in Sample | Purpose / Reference                                      |
|----------------------------|----------------------|-----------------|----------------------------------------------------------|
| <code>v</code>             | Version              | 1               | Generator version, e.g., <code>v ATHENA</code>           |
| <code>j</code>             | Material Descriptor  | 1               | Region material-code table (Section 4.1)                 |
| <code>k</code>             | Global Counts        | 1               | Master header + bounding box (Section 4.2)               |
| <code>m</code>             | Mesh Parameters      | 1               | Purpose minor (Section 4.3)                              |
| <code>c</code>             | Coordinate / Vertex  | 59,685          | <code>c id x y z</code> (Section 5.1)                    |
| <code>e</code>             | Edge                 | 1,752           | <code>e id v1 v2 flag</code> (Section 5.2)               |
| <code>t</code>             | Triangle / Element   | 118,260         | <code>t id region v1 v2 v3 n1 n2 n3</code> (Section 5.3) |
| <code>b</code>             | Boundary Element     | 2,396           | <code>b id</code> — element belongs to current region    |
| <code>r</code>             | Region               | 13              | <code>r id material</code> (Section 5.4)                 |
| <code>s</code>             | Solution Spec        | 1               | <code>s ncodes code...</code> (Section 6.1)              |
| <code>n</code>             | Nodal Solution Value | 60,730          | <code>n id ...</code> (Section 6.2)                      |
| <code>Q,H,I,J,L,W,X</code> | Metadata             | Few             | Region/auxiliary descriptors (Not required)              |

Table 1: Summary of record tags encountered in 2D Silvaco files.

### 3 Units and Coordinate System

Coordinates are provided in microns via TCAD Silvaco ( $\mu\text{m}$ ). Garfield++ uses centimeters (cm), which requires a multiplication factor of  $10^{-4}$ .

The second coordinate axis ( $y$ ) represents depth; it may be negative (indicating surface metal/oxide layers above the silicon) and increases downward into the bulk. Field quantities are provided in the units specified by the quantity code (see Section 7); for example, **ElectricField** components are given in V/cm.

## 4 Header Records

### 4.1 j — Material Descriptor List

```
j 15 4 3 10 9 5 703 704 705 706 711 712 2 700 1 12
|
count(15)      material / region descriptor codes
```

The first integer defines the number of entries. The remaining integers map region material codes (as utilized in **r** and **t** records) to Silvaco’s internal material IDs (the 700-range values, e.g., 703–712, represent specific material identifiers). For field import, this can be parsed to properly label regions with materials.

### 4.2 k — Global Counts and Bounding Box

```
k 59685 60730 1752 13 118260 0 250 -3.6000 50.5000 0 0 1 0 2 0
|      |      |      |      |      |      |      |
nCoord nNode nEdge nReg nTri ? xmax ymin  ymax  ...
```

| Token | Position | Field Value (Sample) | Meaning                               |
|-------|----------|----------------------|---------------------------------------|
|       | 1        | 59685                | Number of <b>c</b> coordinate records |
|       | 2        | 60730                | Number of <b>n</b> nodal records      |
|       | 3        | 1752                 | Number of <b>e</b> edge records       |
|       | 4        | 13                   | Number of <b>r</b> region records     |
|       | 5        | 118260               | Number of <b>t</b> triangle records   |
|       | 6        | 0                    | Reserved / $x_{\min} = 0$             |
|       | 7        | 250                  | $x_{\max}$ ( $\mu\text{m}$ )          |
|       | 8        | -3.6000              | $y_{\min}$ ( $\mu\text{m}$ )          |
|       | 9        | 50.5000              | $y_{\max}$ ( $\mu\text{m}$ )          |
|       | 10+      | 0 0 1 0 2 0          | Reserved / version flags              |

Table 2: Token definitions for the **k** header record.

These counts allow a parser to pre-allocate memory and validate data structures up front.

### 4.3 m — Mesh Parameters

```
m 0 300
```

Two integers representing minor mesh metadata. This information is not required for field reconstruction.

## 5 Geometry Records

### 5.1 c — Coordinate (Vertex)

```
c 1 24.999999999999996 -3.6000000000000001 0
  | |                |                |
  id x(um)            y(um)            z(um, 0 for 2D)
```

A 1-based vertex ID. For 2D simulations, the  $z$  coordinate is always 0.

### 5.2 e — Edge

```
e 1 59434 59685 0
  | |      |      |
  id v1     v2     flag
```

Connects two vertex IDs. Edges describe boundaries and interfaces; they are not strictly required to build the triangulation because triangles reference vertices directly.

### 5.3 t — Triangle (Element)

This is the core topology record for the mesh.

```
t 1 1 29064 29065 29315 2 -1024 58002
  | |
  | |      3 vertex ids    3 neighbour-triangle ids
  | region/material index
  triangle id
```

| Field    | Meaning                                                                   |
|----------|---------------------------------------------------------------------------|
| id       | 1-based triangle ID                                                       |
| region   | Region index (matches <b>r</b> records)                                   |
| v1 v2 v3 | The three vertex IDs (referencing <b>c</b> records)                       |
| n1 n2 n3 | Adjacent triangle IDs; negative numbers indicate a boundary (no neighbor) |

Table 3: Field mappings for the **t** triangle record.

Each triangle specifies its three vertices directly, allowing Garfield++ to build elements without re-triangulating. The neighbor list enables fast point-location via a mesh-walk algorithm during field interpolation.

### 5.4 r — Region

```
r 1 3
r 2 1
r 8 6
```

Defined as **r** <region\_id> <material\_code>. Each **r** line is followed by a contiguous run of **b** records listing the boundary elements belonging to that region. Garfield++ maps each region to a **Medium** object (typically, only silicon regions require an active transport medium).

## 6 Solution Data

Structure (**.str**) and solution (**.sta**) files share an identical mesh structure. The solution block declares its active physical quantities and then lists per-node values.

## 6.1 s — Solution Specification

```
s 7 513 5 151 3 152 71 72
|
count(7)    quantity codes (see Section 7)
```

The first integer specifies the number of quantities; the subsequent integers are quantity codes defining the column order of values within the following **n** records.

In this example: 513 = Region, 5 = Boron, 151 = ActiveBoron, 3 = Phosphorus, 152 = ActivePhosphorus, 71 = DonorConcentration, and 72 = AcceptorConcentration. (This specific layout represents a structure/doping file; electric field codes 120 and 121 appear in the solution **.sta** file).

## 6.2 n — Nodal Value

The verified record layout contains exactly 10 tokens in a fixed format:

```
n <node_id> <field_A> <field_B> <v0> <v1> <v2> <v3> <v4> <v5> <v6>
```

where the 7 trailing values correspond, in sequential order, to the codes declared in the preceding **s** line. Each node ID appears on either one or two **n** lines, distinguished by the value of **field\_A**:

- **Data line** (**field\_A** != 0; values {1,3,6} observed): The 7 value columns contain the real solution data for the node. Every node has exactly one such line.
- **Flag line** (**field\_A** == 0): A boundary/interface marker. **field\_B** carries an identifying code (e.g., -1022), and the 7 value columns are all zero. These lines contain no field data and must be skipped.

**Parser Rule:** For each **n** line, read **field\_A** (the 3rd token). If it is 0, skip the line. Otherwise, the solution values begin at the 5th token (immediately following **field\_B**).

Node IDs are 0-based, whereas coordinate IDs in **c** records are 1-based; they map directly via:

$$\text{coord\_id} = \text{node\_id} + 1 \quad (1)$$

## 7 Quantity Codes

## 8 Parsing Recipe for ComponentSilvaco::Initialise

1. Read the **k** record to extract entity counts (**nCoord**, **nNode**, **nEdge**, **nReg**, **nTri**) and the spatial bounding box.
2. Read all **c** records into a vertex array, transforming coordinates from  $\mu\text{m}$  to  $\text{cm}$  ( $x_{\text{cm}} = x_{\mu\text{m}} \times 10^{-4}$ ).
3. Read all **t** records into an element array: store the 3 vertex indices, the region index, and the 3 neighbor indices.
4. Read **r** records to assemble the region list, mapping material codes via the **j** table, and associate active regions with a Garfield++ **Medium**.
5. (Optional) Read **e** records if boundary descriptions are needed.
6. Read the **s** record to determine the quantity column layout, then parse all **n** records into a per-node value table while applying the coordinate offset ( $\text{coord\_id} = \text{node\_id} + 1$ ).

| Code | Observable                | Units            |
|------|---------------------------|------------------|
| 513  | Region                    | —                |
| 5    | Boron                     | cm <sup>-3</sup> |
| 151  | ActiveBoron               | cm <sup>-3</sup> |
| 3    | Phosphorus                | cm <sup>-3</sup> |
| 152  | ActivePhosphorus          | cm <sup>-3</sup> |
| 71   | DonorConcentration        | cm <sup>-3</sup> |
| 72   | AcceptorConcentration     | cm <sup>-3</sup> |
| 115  | NetDoping                 | cm <sup>-3</sup> |
| 149  | TotalDoping               | cm <sup>-3</sup> |
| 100  | ElectrostaticPotential    | V                |
| 103  | ElectricFieldMagnitude    | V/cm             |
| 120  | ElectricField_x ( $E_x$ ) | V/cm             |
| 121  | ElectricField_y ( $E_y$ ) | V/cm             |
| 124  | LatticeTemperature        | K                |

Table 4: Mapping of Silvaco quantity codes to physical observables.

To evaluate the field at an arbitrary point  $(x, y)$ , locate the containing triangle utilizing the neighbor links for a fast mesh-walk search, then evaluate the field via barycentric interpolation of the three nodal values at the triangle’s vertices.

## 9 Two Dialects: ATHENA .str vs ATLAS .sta

Silvaco utilizes two related variants of this format, distinguished by the **v** line and the comment banner:

| Feature         | ATHENA .str                                  | ATLAS .sta                                        |
|-----------------|----------------------------------------------|---------------------------------------------------|
| <b>v</b> line   | <b>v</b> ATHENA                              | <b>v</b> ATLAS                                    |
| Banner          | #silvaco.file ASCII ... contents=SDB,Coord2D | #SSF 2.0 generic                                  |
| Typical Content | Geometry + doping profiles                   | Device solution: full field solution              |
| <b>k</b> Header | Mesh counts + bounding box (all integers)    | Solution metadata (contains floats)               |
| Mesh Topology   | Full (c/e/t/r)                               | Full (c/e/t/r) — re-declared inline               |
| <b>s</b> Codes  | Doping quantities ( $\sim 7$ )               | Full device output ( $\sim 35$ , inc. $E_x/E_y$ ) |

Table 5: Comparison between ATHENA structural files and ATLAS solution files.

### Parser Implications:

- Detect the dialect first via the **v** token or the banner text. Do not assume the **k** line contains mesh counts in ATLAS files, as it represents solution metadata where the third field is a float (which will crash a rigid integer parser).
- When parsing an ATLAS file where **k** is not a mesh header, compute the entity counts dynamically by counting the **c**, **t**, **e**, and **r** records directly during a preliminary pass.
- The ATLAS .sta file is entirely self-contained: it carries both the full mesh and the field solutions. For electric field import, only the .sta file is required.
- The **n** records in an ATLAS .sta file carry two integer prefix fields after the node ID (**n** <id> <p1> <p2> <val0> <val1> ...), meaning the value columns begin at the 4th token — matching the effective data offset used in the ATHENA .str parser logic.

Listing 1: Example ATLAS .sta headers

```
v ATLAS
#SSF 2.0 generic
j 4 1 2 94 95
k 2 2 0.5528019173891893 1.08          <- solution metadata, not mesh counts
c 1 25 -3.6 0
...
s 35 513 115 149 71 72 69 70 100 103 120 121 301 106 111 ...   <- 120/121 = field
n 0 6 8 0.0e+00 0.0e+00 ...          <- 35 value columns per node
```

## 10 Conclusion

Because triangles carry vertex indices and neighbor links directly, the mesh maps directly onto Garfield++’s existing TCAD structures without requiring external Delaunay tools. The ATLAS .sta format provides a self-contained target for a `ComponentSilvaco::Initialise(staFile)` entry point.